

Isotope tracing for Apache Kafka

Lightweight, end-to-end record tracing for Kafka built entirely on **public extension points** — a **ProducerInterceptor** plus record headers. No broker changes, no agent, no vendor lock-in. Stamp a UUIDv7 trace id and origin metadata on first produce, accumulate a hop on every re-produce, and reconstruct latency / topology / hop-distribution from the headers (or from the optional Prometheus meters).

Two artifacts, so you only take what you need:

Module	Coordinate	Pulls	Use it for
kafka-isotope-core	<code>ai.signalroom:kafka-isotope-core</code>	Jackson, SLF4J (<code>kafka-clients</code> is <code>compileOnly</code>)	Trace propagation — the interceptor, headers, consume markers.
kafka-isotope-metrics	<code>ai.signalroom:kafka-isotope-metrics</code>	kafka-isotope-core + Micrometer/Prometheus	Optional <code>/metrics</code> exporter for the stateless reports.
kafka-isotope-otel	<code>ai.signalroom:kafka-isotope-otel</code>	kafka-isotope-core + OTLP protobuf	Optional span writer — one OTLP span per hop on a Kafka topic.

`kafka-isotope-core` has no metrics or tracing dependency. Emission is routed through the `IsotopeMetricsSink` and `IsotopeSpanSink` interfaces, which start out as no-ops. Adding `kafka-isotope-metrics` or `kafka-isotope-otel` to the classpath doesn't change that: the real sink is registered only when your application calls `PrometheusIsotopeMetrics.start(port)` or `KafkaOtlpSpanSink.start(config)` (steps 3 and 4 below), and stays a no-op if that call fails. Until then, each record costs an "is it enabled?" check and no metric or span work.

Install (Gradle, GitHub Packages)

```
repositories {
    maven { url 'https://maven.pkg.github.com/j3-signalroom/kafka-isotope' }
}
dependencies {
    implementation 'ai.signalroom:kafka-isotope-core:0.19.0'
    implementation 'ai.signalroom:kafka-isotope-metrics:0.19.0' // optional
    // only for Prometheus
    implementation 'ai.signalroom:kafka-isotope-otel:0.19.0' // optional
    // only for OTel spans
}
```

Use it

1. Produce side — register the interceptor. Every `send()` is stamped/hopped automatically; nothing else to call.

```
props.put(ProducerConfig.INTERCEPTOR_CLASSES_CONFIG,
          IsotopeProducerInterceptor.class.getName());
props.put(IsotopeProducerInterceptor.SERVICE_NAME_CONFIG, "order-intake-
service");
props.put(IsotopeProducerInterceptor.PIPELINE_NAME_CONFIG, "orders");
```

2. Consume side — adopt to continue the trace, or mark a terminal consume.

```
// A stage that consumes then re-produces ("hop"): adopt so the next send()
// continues the same trace instead of starting a new one.
IsotopeContext.adoptFromRecord(record, "order-enrichment-service");
// ... process and produce downstream; the interceptor reads the adopted
context ...
IsotopeContext.clear(); // per-record, on the same thread

// A terminal consumer (no re-produce): write a bipartite consume-edge
marker.
IsotopeContext.recordConsume(record, "shipping-notification-service",
markerProducer);
```

3. (Optional) Metrics — start the Prometheus exporter once at boot.

```
import ai.signalroom.kafka.isotope.metrics.PrometheusIsotopeMetrics;

PrometheusIsotopeMetrics.start(9404); // serves GET /metrics; no-op on a
port clash
```

This serves `isotope_hop_latency_*`, `isotope_hop_records_total`, and the consume-side meters at `http://localhost:9404/metrics`. Until you call `start`, the interceptor's metric calls are inert. See [kafka-isotope-metrics/README.md](https://github.com/signalroom/kafka-isotope-metrics/blob/main/README.md).

4. (Optional) Spans — start the OTLP span writer once at boot.

```
import ai.signalroom.kafka.isotope.otel.KafkaOtlpSpanSink;

KafkaOtlpSpanSink.start(Map.of(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG,
" kafka:9092"));
```

Every hop then lands on `isotope_trace_spans` as an OTLP span, ready for a stock OpenTelemetry Collector. Queued off the hot path and dropped rather than allowed to delay a `send()`. See [kafka-isotope-otel/README.md](#).

What's where

- `Isotope` — the header model + JSON codec, UUIDv7, `MAX_HOPS`.
- `IsotopeContext` — thread-local context, `adoptFromRecord`, `recordConsume`.
- `IsotopeProducerInterceptor` — the `ProducerInterceptor` that stamps and hops.
- `IsotopeMetrics` / `IsotopeMetricsSink` — the metrics seam (core stays metrics-free).
- `IsotopeSpans` / `IsotopeSpanSink` — the span seam (core stays tracing-free).

For the full meter/PromQL reference and the design rationale (why only three of seven reports are metrics-native), see [docs/metrics.md](#) and [docs/design.md](#). The runnable demo and one-command Prometheus + Grafana showcase live in the `app` module and [k8s/monitoring](#).